Bitcoin Switch Firmware – ESP32 + BTCPay

Oct 14, 2026 · @Bongani

Get the code

This ESP32 sketch turns paid Lightning invoices on BTCPay Server into arcade credits by clicking a relay on the coin input.

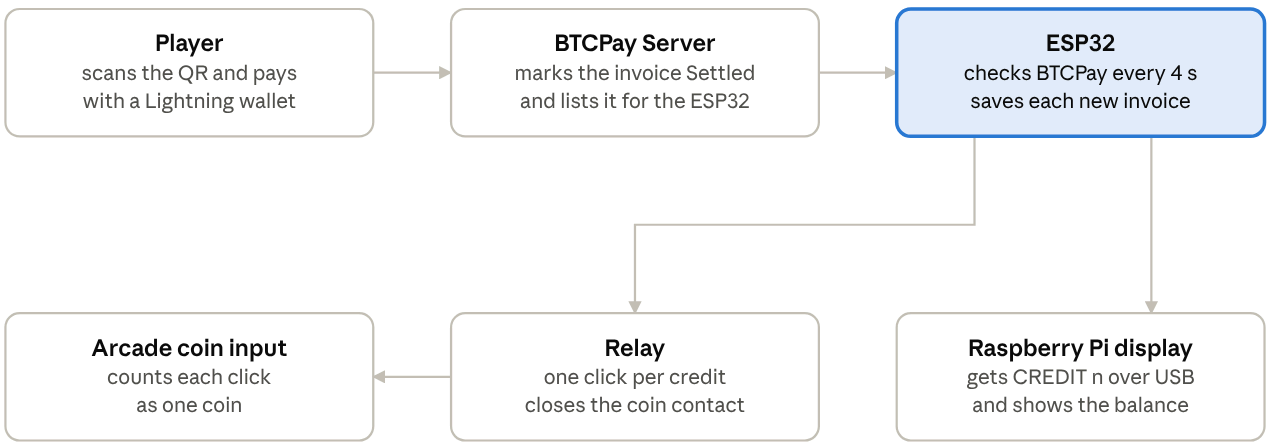
Get the code: copy the whole sketch from Full code at the end of this page and paste it into a new Arduino sketch named `bitcoin_switch`.

The settings at the top hold placeholders only. Don't share a copy that has your Wi-Fi password or API key filled in.

What it does

The ESP32 asks BTCPay for paid invoices every 4 seconds and turns each new one into coin clicks.

Every paid invoice clicks the coin relay once per credit



payment to coin · 6 parts

The ESP32 saves each invoice before the relay clicks, so a reboot can never pay the same invoice out twice.

What you need

One ESP32, one relay and a BTCPay store with Lightning are all the switch needs.

Item	Details
ESP32 dev board	Any ESP32 DevKit. Select Node32s or ESP32 Dev Module in Arduino IDE.
Relay module	1 channel, 5 V. Its contact stands in for the coin switch.
USB cable	Must carry data. Connects the ESP32 to the Raspberry Pi.
Arduino IDE	1.8.19 (2.x works too)
ESP32 board package	ESP32 Arduino core 2.0.14, from Boards Manager
Library	ArduinoJson 7.x by Benoit Blanchon, from Library Manager
BTCPay Server	A store with Lightning enabled and an API key

BTCPay setup

The ESP32 only reads invoices, so its API key needs just one permission: **View invoices**.

1. Create a store just for the arcade, with Lightning enabled. Turn off on-chain payments, because confirmations take too long for a game.
2. In that store, add a **Point of Sale** app with one item, for example "1 Credit" at R5.00. Its link (`https://your-server/apps/<APP_ID>/pos`) is what players scan.
3. Go to **Account → API Keys → Generate Key**, tick only **View invoices** for that store, and copy the key.
4. Copy the **Store ID** from Store Settings → General.

If the store also sells other things, set `ORDER_ID_FILTER` to `pos-app_<APP_ID>` so only the arcade's Point of Sale counts.

Flash it

Fill in the settings at the top of the sketch, then upload it like any other ESP32 sketch.

Setting	What to put	Example
`WIFI_SSID`, `WIFI_PASSWORD`	Your Wi-Fi name and password	
`BTCPAY_URL`	Your server address, with no slash at the end	`https://btcpay.example.com`
`STORE_ID`	From Store Settings → General	
`API_KEY`	The View invoices key	
`CREDIT_PRICE`	Price of one credit, in the store currency	`5.00`
`RELAY_ACTIVE_LOW`	`true` if your relay switches on when IN is LOW	`false`

1. Sketch → Include Library → Manage Libraries → install **ArduinoJson** version 7.x.
2. Tools → Board → **Node32s** (or ESP32 Dev Module), then pick the COM port.
3. Click **Upload**.

If the upload stops with "Failed to connect… waiting for packet header", hold the **BOOT** button while it says "Connecting…". If that doesn't fix it, set Tools → Upload Speed to 115200 and use a USB cable that carries data.

Wiring

Each relay click is one coin, so the relay contact goes in parallel with the arcade's coin switch.

From	To	Note
ESP32 GPIO 13	Relay IN	To use another pin, change `RELAY_PIN`. Avoid GPIO 0, 2, 12 and 15.
ESP32 VIN (5 V)	Relay VCC	
ESP32 GND	Relay GND	
Relay COM and NO	The arcade's two coin-input wires	In parallel with the coin button or coin acceptor

From	To	Note
ESP32 USB	Raspberry Pi USB	Powers the ESP32 and sends credit messages to the display

Test and troubleshoot

Open the Serial Monitor at 115200 baud. `HB OK` every 5 seconds means the switch is online and watching BTCPay.

Message	Meaning
`WIFI OK <ip>`	Joined Wi-Fi
`HB OK`, `HB NOWIFI`, `HB NOTIME`, `HB APIERR`	Heartbeat every 5 s: online, no Wi-Fi, clock not synced yet, can't reach BTCPay
`CREDIT 2 <invoiceId>`	A payment for 2 credits. The relay clicks twice.
`INFO first start: skipped …`	On its very first start, the switch ignores invoices that were already paid
`ERR BTCPay replied 401` (or 403, 404)	The Store ID or the API key permission is wrong

Type `TEST` and press Enter: the relay clicks once and `CREDIT 1 TEST` appears.

Problem	Fix
Relay stays on at boot, or clicks the wrong way round	Set `RELAY_ACTIVE_LOW = true`
Game misses some coins	Set `PULSE_MS = 150` and `PULSE_GAP_MS = 400`
`HB APIERR` keeps showing	Read the `ERR` line printed just before it

Full code

Copy everything in the box into a new Arduino sketch named `bitcoin_switch`, then fill in the settings section.

```
/*
  Bitcoin Ekasi Arcade - Bitcoin switch firmware (ESP32 + BTCPay Server)
  ----------------------------------------------------------------------
  Watches a BTCPay Server store for paid ("Settled") invoices. For every new
  paid invoice it:
    1. works out how many credits were bought (paid amount / CREDIT_PRICE)
    2. remembers the invoice in flash, so it can never be paid out twice,
       even after a reboot or power cut
    3. pulses the relay once per credit (the relay closes the arcade's coin
input)
    4. tells the Raspberry Pi over USB serial:  CREDIT <n> <invoiceId>

  Messages sent to the Pi (115200 baud, one per line):
    BOOT ...           firmware started
    WIFI OK <ip>       connected to Wi-Fi
    CREDIT <n> <id>    n credits were paid for
    HB <state>         heartbeat every 5 s; state = OK | NOWIFI | NOTIME |
APIERR
    INFO / ERR ...     extra detail, handy in the Arduino Serial Monitor
  Commands it accepts (from the Pi, or typed into the Serial Monitor):
    TEST               pulse the relay once and send "CREDIT 1 TEST" (wiring
check)

  Board:    Node32s (or "ESP32 Dev Module"), ESP32 Arduino core 2.0.x
  Library:  ArduinoJson 7.x by Benoit Blanchon (Sketch > Include Library >
Manage Libraries)
            Everything else (WiFi, HTTPClient, Preferences) comes with the
ESP32 core.
            The old LNbits libraries (WebSockets, PageBuilder) are NOT
needed.
*/

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <Preferences.h>
#include <ArduinoJson.h>
#include <time.h>

#if ARDUINOJSON_VERSION_MAJOR < 7
#error "This sketch needs ArduinoJson 7.x - update it in Sketch > Include
Library > Manage Libraries"
#endif
```

```
// ================================== SETTINGS
==================================

// Wi-Fi
const char* WIFI_SSID     = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// BTCPay Server
const char* BTCPAY_URL = "https://YOUR-BTCPAY-SERVER";  // no slash at the
end
const char* STORE_ID   = "YOUR_STORE_ID";               // Store Settings >
General > Store ID
const char* API_KEY    = "YOUR_API_KEY";                // Account > API
Keys, permission "View invoices" only

// Only count invoices from one Point of Sale app (use this if the store
sells other things).
// Format: "pos-app_" + the app ID from the POS link (.../apps/<APP_ID>/pos).
// Leave "" to count every paid invoice in the store.
const char* ORDER_ID_FILTER = "";

// Credits
const float CREDIT_PRICE = 5.00;           // price of ONE credit in the
store currency (e.g. R5.00)
                                           // 0 = every paid invoice gives
exactly 1 credit
const int   MAX_CREDITS_PER_PAYMENT = 20;

// Relay
const int  RELAY_PIN        = 13;     // GPIO -> relay IN. Avoid 0, 2, 12, 15
(boot pins)
const bool RELAY_ACTIVE_LOW = false;  // set true if your relay clicks ON
when IN is LOW
const int  PULSE_MS         = 100;    // how long the "coin" contact stays
closed
const int  PULSE_GAP_MS     = 300;    // pause between coins

const int  LED_PIN = 2;               // onboard LED: on = Wi-Fi and BTCPay
OK. -1 to disable

// Polling
const unsigned long POLL_INTERVAL_MS = 4000;
```

```
const long LOOKBACK_SECONDS = 3600;   // check invoices created in the last
hour

// HTTPS: nullptr = encrypted, but the server certificate is not checked.
// Paste your server's root CA certificate (PEM) here to have it checked.
const char* ROOT_CA = nullptr;

//
================================================================================

const int RELAY_ON  = RELAY_ACTIVE_LOW ? LOW : HIGH;
const int RELAY_OFF = RELAY_ACTIVE_LOW ? HIGH : LOW;

// Invoices already paid out, stored as 64-bit hashes of the invoice ID.
// Must hold more entries than the store could settle within
LOOKBACK_SECONDS.
const int SEEN_SLOTS = 200;
uint64_t seen[SEEN_SLOTS];
uint16_t seenNext = 0;
bool baselineDone = false;   // first boot: existing invoices are skipped,
not credited
Preferences prefs;

enum State { ST_NOWIFI, ST_NOTIME, ST_APIERR, ST_OK };
State state = ST_NOWIFI;
bool lastPollOk = false;
bool wifiWasUp = false;
unsigned long lastPoll = 0, lastHeartbeat = 0, wifiLostAt = 0;
String serialLine;

// ------------------------------------------------------------ remembering
--

uint64_t idHash(const char* s) {          // FNV-1a
  uint64_t h = 14695981039346656037ULL;
  while (*s) { h ^= (uint8_t)*s++; h *= 1099511628211ULL; }
  return h ? h : 1;                       // 0 marks an empty slot
}

bool isSeen(const char* id) {
  uint64_t h = idHash(id);
  for (int i = 0; i < SEEN_SLOTS; i++) if (seen[i] == h) return true;
  return false;
```

```
}

void saveSeen() {
  size_t a = prefs.putBytes("seen", seen, sizeof(seen));
  prefs.putUShort("next", seenNext);
  if (a != sizeof(seen)) Serial.println("ERR could not save paid-invoice list
to flash");
}

void rememberInvoice(const char* id, bool save) {
  seen[seenNext] = idHash(id);
  seenNext = (seenNext + 1) % SEEN_SLOTS;
  if (save) saveSeen();
}

void loadSeen() {
  memset(seen, 0, sizeof(seen));
  prefs.begin("arcade", false);
  if (prefs.getBytesLength("seen") == sizeof(seen)) {
    prefs.getBytes("seen", seen, sizeof(seen));
    seenNext = prefs.getUShort("next", 0) % SEEN_SLOTS;
    baselineDone = prefs.getBool("baseline", false);
  } else {
    baselineDone = false;   // nothing stored yet (or SEEN_SLOTS changed):
start fresh
  }
}

// ------------------------------------------------------------------ relay -
-

void pulseRelay(int times) {
  for (int i = 0; i < times; i++) {
    digitalWrite(RELAY_PIN, RELAY_ON);
    delay(PULSE_MS);
    digitalWrite(RELAY_PIN, RELAY_OFF);
    delay(PULSE_GAP_MS);
  }
}

// ------------------------------------------------------------------ BTCPay -
-
```

```
float jsonNumber(JsonVariant v) {         // BTCPay sends amounts as strings,
e.g. "5.00"
  if (v.is<const char*>()) return atof(v.as<const char*>());
  return v.as<float>();
}

int creditsFor(JsonObject inv) {
  if (CREDIT_PRICE <= 0) return 1;
  float amount = jsonNumber(inv["amount"]);
  float paid   = jsonNumber(inv["paidAmount"]);
  float value  = paid > amount ? paid : amount;
  int credits  = (int)floor(value / CREDIT_PRICE + 0.05f);  // 5% slack for
rounding
  if (credits < 1) credits = 1;           // it is Settled, so it was paid in
full
  if (credits > MAX_CREDITS_PER_PAYMENT) credits = MAX_CREDITS_PER_PAYMENT;
  return credits;
}

// Returns true if BTCPay answered properly.
bool pollBtcpay() {
  time_t now = time(nullptr);
  String url = String(BTCPAY_URL) + "/api/v1/stores/" + STORE_ID +
               "/invoices?status=Settled&take=100&startDate=" + String((long)
(now - LOOKBACK_SECONDS));
  if (ORDER_ID_FILTER[0]) { url += "&orderId="; url += ORDER_ID_FILTER; }

  WiFiClientSecure client;
  if (ROOT_CA) client.setCACert(ROOT_CA); else client.setInsecure();

  HTTPClient http;
  http.useHTTP10(true);                   // no chunked replies, so the JSON
can be streamed
  http.setTimeout(15000);
  if (!http.begin(client, url)) { Serial.println("ERR check BTCPAY_URL");
return false; }
  http.addHeader("Authorization", String("token ") + API_KEY);
  http.addHeader("Accept", "application/json");

  int code = http.GET();
  if (code != 200) {
    Serial.printf("ERR BTCPay replied %d%s\n", code,
                  (code == 401 || code == 403 || code == 404) ? " - check
```

```
STORE_ID and API_KEY permissions" : "");
    http.end();
    return false;
  }

  // Keep only the fields we need, so a long invoice list fits in memory.
  JsonDocument filter;
  filter[0]["id"] = true;
  filter[0]["amount"] = true;
  filter[0]["paidAmount"] = true;

  JsonDocument doc;
  DeserializationError err = deserializeJson(doc, http.getStream(),
DeserializationOption::Filter(filter));
  http.end();
  if (err) { Serial.printf("ERR reading BTCPay reply: %s\n", err.c_str());
return false; }

  JsonArray invoices = doc.as<JsonArray>();
  if (invoices.isNull()) { Serial.println("ERR unexpected BTCPay reply");
return false; }

  if (!baselineDone) {
    // Very first run: mark what is already paid as done, so old invoices
don't pay out.
    for (JsonObject inv : invoices) {
      const char* id = inv["id"] | "";
      if (id[0] && !isSeen(id)) rememberInvoice(id, false);
    }
    saveSeen();
    prefs.putBool("baseline", true);
    baselineDone = true;
    Serial.printf("INFO first start: skipped %d invoices that were already
paid\n", (int)invoices.size());
    return true;
  }

  // BTCPay lists newest first, so walk backwards to pay out oldest first.
  for (int i = (int)invoices.size() - 1; i >= 0; i--) {
    JsonObject inv = invoices[i];
    const char* id = inv["id"] | "";
    if (!id[0] || isSeen(id)) continue;
```

```
    int credits = creditsFor(inv);
    // Save BEFORE clicking the relay: if the relay causes a brown-out and
the
    // ESP32 reboots, this invoice is already marked and can't pay out twice.
    rememberInvoice(id, true);
    Serial.printf("CREDIT %d %s\n", credits, id);
    pulseRelay(credits);
  }
  return true;
}

// ------------------------------------------------------------ connections -
-

bool timeIsSet() { return time(nullptr) > 1700000000; }   // after Nov 2023 =
NTP has synced

void handleWifi() {
  bool up = WiFi.status() == WL_CONNECTED;
  if (up && !wifiWasUp) {
    Serial.printf("WIFI OK %s\n", WiFi.localIP().toString().c_str());
  } else if (!up && wifiWasUp) {
    Serial.println("WIFI DOWN");
    wifiLostAt = millis();
  } else if (!up && millis() - wifiLostAt > 30000) {
    WiFi.disconnect();                    // auto-reconnect gave up: start
again
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    wifiLostAt = millis();
  }
  wifiWasUp = up;
}

void updateState() {
  if (WiFi.status() != WL_CONNECTED) state = ST_NOWIFI;
  else if (!timeIsSet())             state = ST_NOTIME;
  else                               state = lastPollOk ? ST_OK : ST_APIERR;
  if (LED_PIN >= 0) digitalWrite(LED_PIN, state == ST_OK ? HIGH : LOW);
}

const char* stateName() {
  switch (state) {
    case ST_OK:     return "OK";
```

```
    case ST_NOTIME: return "NOTIME";
    case ST_APIERR: return "APIERR";
    default:        return "NOWIFI";
  }
}

void readSerialCommands() {
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') {
      serialLine.trim();
      if (serialLine.equalsIgnoreCase("TEST")) {
        Serial.println("CREDIT 1 TEST");
        pulseRelay(1);
      }
      serialLine = "";
    } else if (serialLine.length() < 32) {
      serialLine += c;
    }
  }
}

// ------------------------------------------------------------------ main --

void setup() {
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, RELAY_OFF);
  if (LED_PIN >= 0) pinMode(LED_PIN, OUTPUT);

  Serial.begin(115200);
  delay(300);
  Serial.println();
  Serial.println("BOOT bitcoin_switch 1.0 (BTCPay)");

  loadSeen();

  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  wifiLostAt = millis();

  configTime(0, 0, "pool.ntp.org", "time.google.com", "time.cloudflare.com");
```

```
}

void loop() {
  readSerialCommands();
  handleWifi();

  if (millis() - lastPoll >= POLL_INTERVAL_MS) {
    lastPoll = millis();
    if (WiFi.status() == WL_CONNECTED && timeIsSet()) lastPollOk =
pollBtcpay();
    else lastPollOk = false;
  }

  updateState();

  if (millis() - lastHeartbeat >= 5000) {
    lastHeartbeat = millis();
    Serial.printf("HB %s\n", stateName());
  }

  delay(10);
}
```